Árvores Rubro-Negras

PROF. RICARDO P. MESQUITA

Árvore Rubro-Negra

A árvore rubro-negra tem esse nome devido a "**coloração**" de seus nós

Uma árvore rubro negra (ARN) é uma árvore binária de busca com um campo adicional que armazena se o nó é rubro ou negro

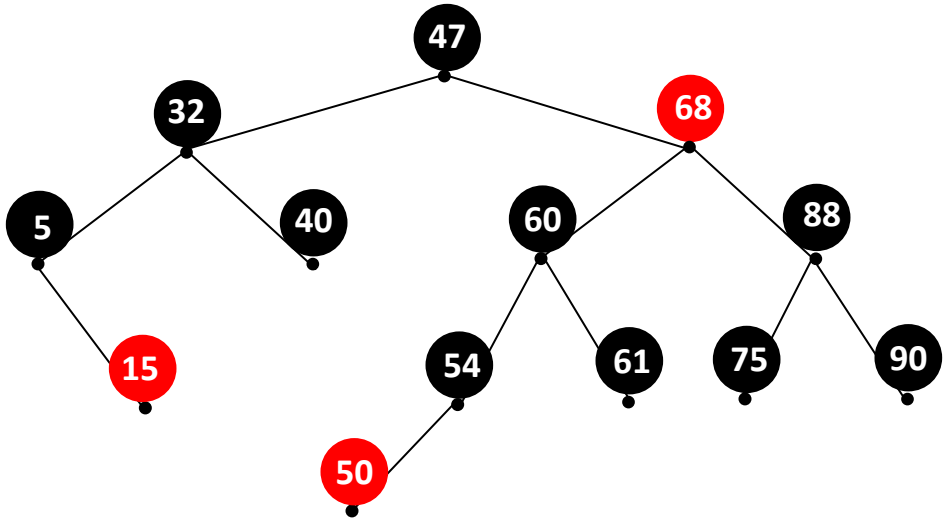
O fato de um nó ser rubro ou negro é usado como fator de balanceamento da ARN

Propriedades

1. Cada nó tem uma cor que é rubro ou negro. Por convenção, uma árvore não vazia (ou subárvore) tem a cor de sua raiz e uma árvore vazia é negra.
2. A raiz é negra.
3. Qualquer caminho da raiz até uma subárvore vazia tem um número igual de nós negros.
4. As subárvores de um nó rubro são sempre negras.

Conceitos

(Altura negra) A altura negra de uma árvore rubro-negra A, denotada an(A) é o número de nós negros que se encontram nos caminhos da raiz até uma folha



Observe que, pela terceira condição da definição de árvore rubro-negra, esse número é bem definido. No caso da árvore acima, a altura negra é 3

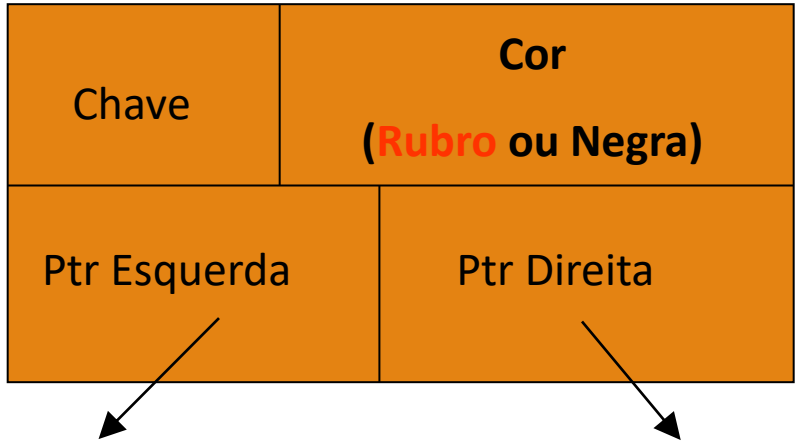
Conceitos

A altura de qualquer árvore rubro-negra é logarítmica no número de chaves armazenadas

A busca nas árvores rubro-negra tem complexidade logarítmica.

Uma ARN impede que uma subárvore fique com o dobro da altura da outra subárvore de um nó.

Estrutura interna de um nó:



Inserção

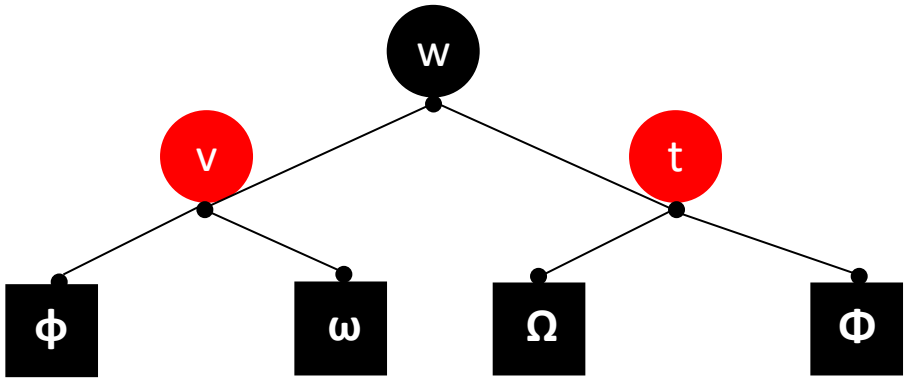
Todo nó a ser inserido por convenção é rubro (pois se fosse negro não seguiria a propriedade 3).

Se após a inserção for quebrada qualquer propriedade da ARN devem ser feitas rotações e/ou inversão de cores dos nós para que sejam satisfeitas todas as propriedades.

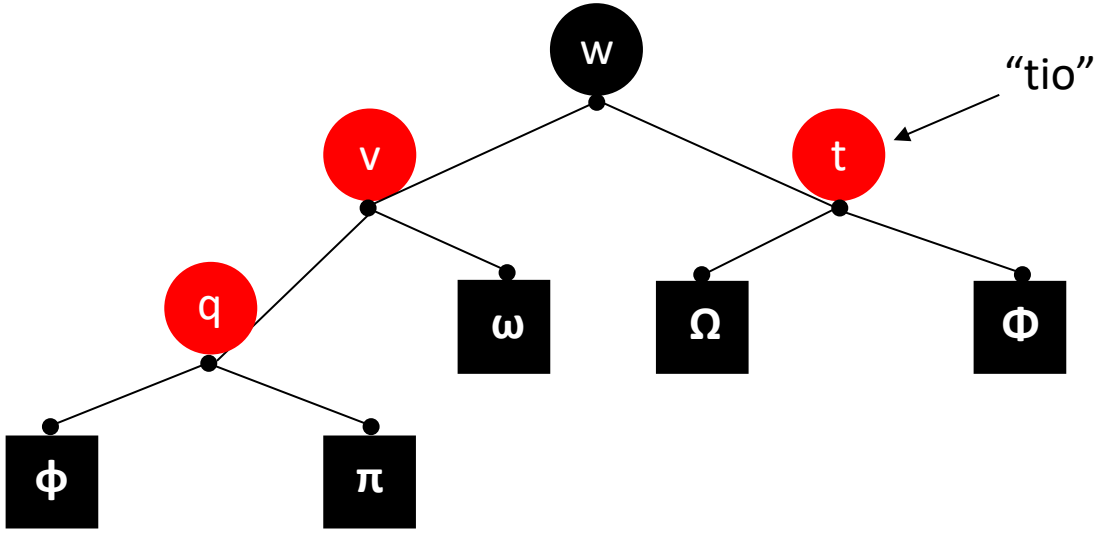
As regras de inserção levam em consideração a cor do “tio” (o outro filho do pai do nó que recebeu o novo nó) do nó inserido.

Situações

Considere a árvore:



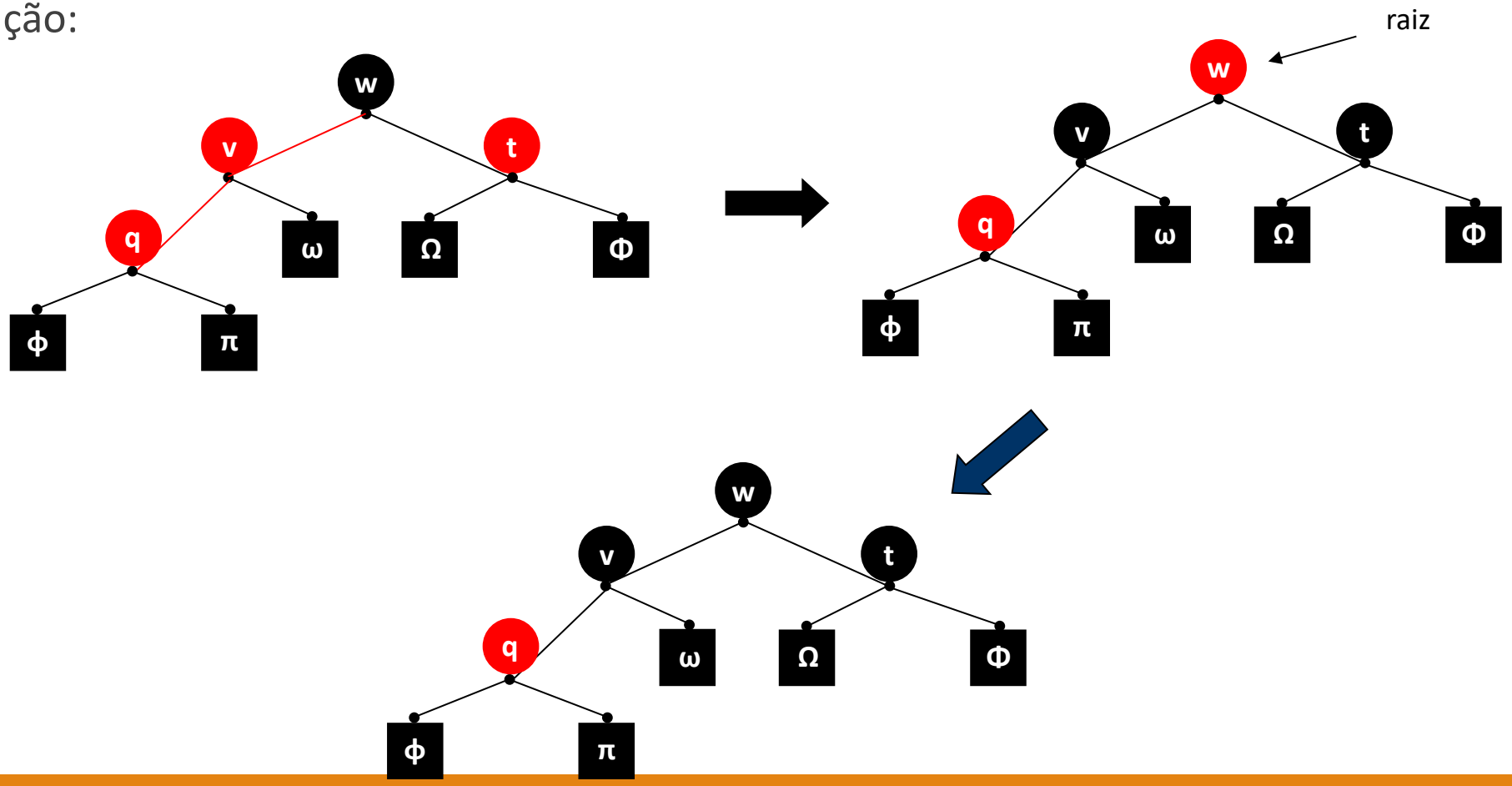
- Inserção de q:



- <u>Se ***t*** for rubro</u>: o pai de ***t*** torna-se rubro e, os filhos de ***w*** tornam-se negros. Caso ***w*** seja a raiz, basta trocar sua cor para negro.

Situações

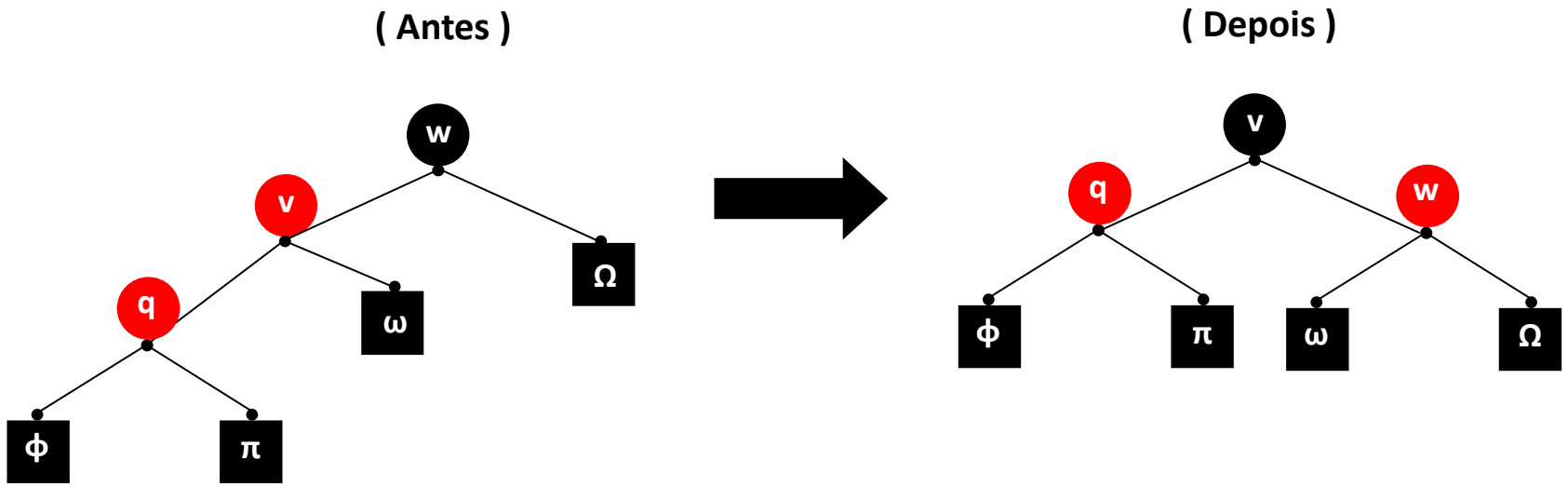
Execução:



Situações

Se não há um tio: neste caso faz-se uma operação de ***rotação*** e, se necessário, uma inversão de cores. Há 4 subcasos a considerar:

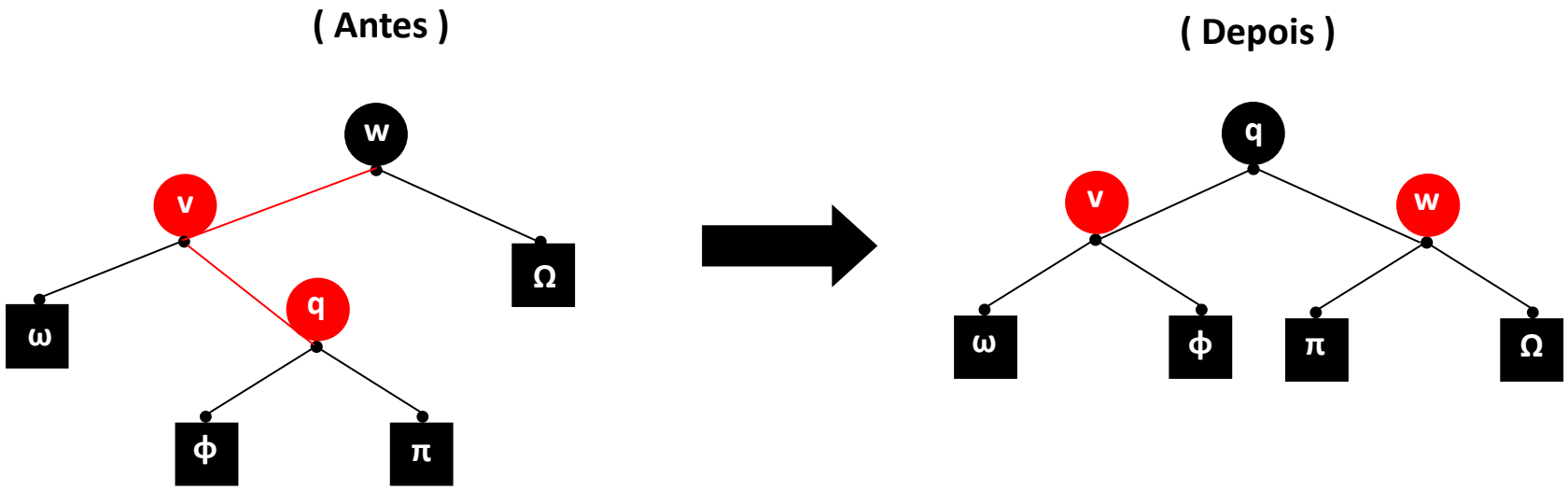
1. Se ***q*** é filho esquerdo de ***v*** e ***v*** é filho esquerdo de ***w*** é realizada uma rotação simples a direita e as cores de ***v*** e ***w*** são invertidas.



Situações

Se não há um tio: neste caso faz-se uma operação de ***rotação*** e, se necessário, uma inversão de cores. Há 4 subcasos a considerar:

2. Se ***q*** é filho direito de ***v*** e ***v*** é filho esquerdo de ***w*** é realizada uma rotação dupla a direita e as cores de ***q*** e ***w*** são invertidas.



Situações

Se não há um tio: neste caso faz-se uma operação de ***rotação*** e, se necessário, uma inversão de cores. Há 4 subcasos a considerar:

Os 3º e 4º casos são simétricos aos casos anteriores.

Complexidade da Inserção

A complexidade da inserção, que é a da inserção em árvore binária de busca, é logarítmica.

O pior caso da fase de balanceamento é se tiver que aplicar a inversão de cores até a raiz.

Como o tamanho do caminho da raiz até qualquer folha é logarítmico, o número de operações também é logarítmico.

Conclusão: a complexidade da inserção em árvores rubro-negras é logarítmica.

Remoção

A remoção em árvores rubro-negras pode ser realizada também com um número logarítmico de operações.

O procedimento de remoção é composto de uma etapa de remoção em árvore binária de busca seguido de uma etapa de balanceamento, caso as propriedades rubro-negras teriam sido destruídas durante a operação.

Remoção

Se o nó removido for rubro, a árvore continua rubro-negra, pois todas as condições da definição ficam válidas:

1. Os nós resultantes tem cor rubro ou negro.
2. A raiz, que era negra, não foi removida.
3. Nenhum nó negro foi removido, portanto todos os caminhos da raiz até uma folha tem um número igual de nós negros.
4. Os filhos de todos os nós rubros não removidos não foram alterados e portanto ficam negros.

Remoção

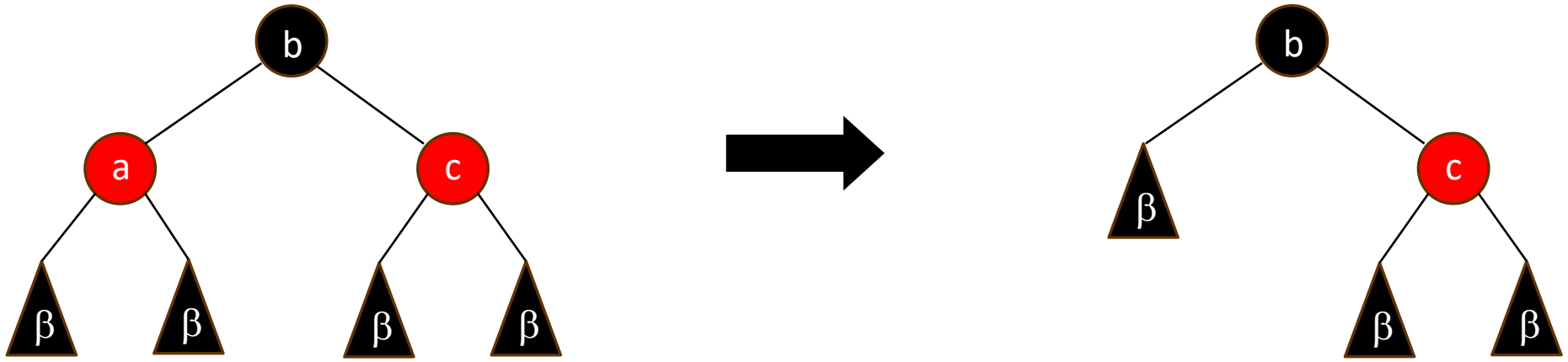
Se o nó removido for negro, o número de nós de pelo menos um caminho foi decrementado e consequentemente a terceira condição ficou inválida. Quando isto acontece, dois tipos de solução são possíveis:

- **remoção preguiçosa-** A remoção preguiçosa consiste em marcar o nó como removido, mas sem tira-lo da árvore. Nenhum remanejamento é necessário. Em compensação, os algoritmos de inserção e busca devem ser modificados para levar em conta que alguns nós da árvore devem ser considerados como ausentes. A adoção desta solução é possível quando as árvores rubro-negras são usadas no contexto de uma aplicação com poucas operações de remoção.
- **remoção efetiva -** Através de um número logarítmico de operações, a remoção efetiva restabelece as propriedades para que a árvore seja rubro-negra. Essas operações são detalhadas em seguida.

Remoção Efetiva

Caso o nó a ser removido seja ***rubro,*** as propriedades da ARN não são afetadas.

Por exemplo, remover o nó **a**:



Remoção Efetiva

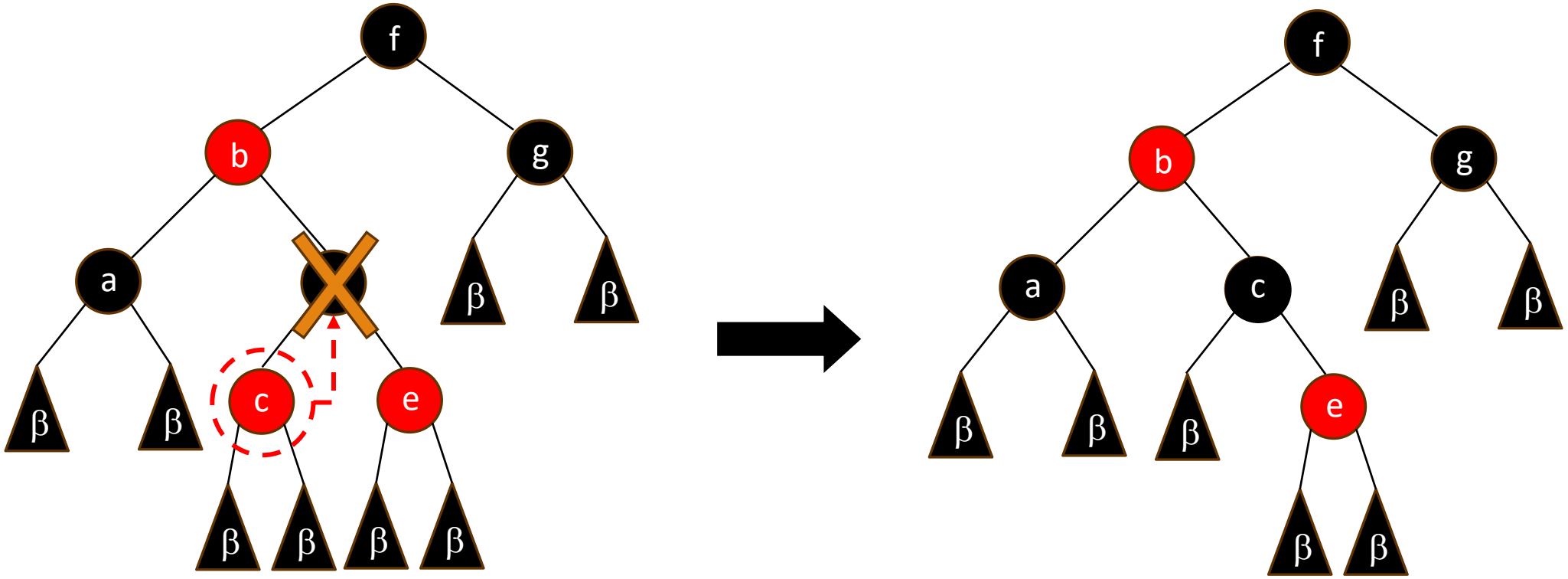
Quando o nó a remover é negro, todos os caminhos da raiz até uma folha passando por esse nó terão um nó negro a menos.

Seja ***c*** o nó que passar a ocupar a posição de ***y*** na árvore.

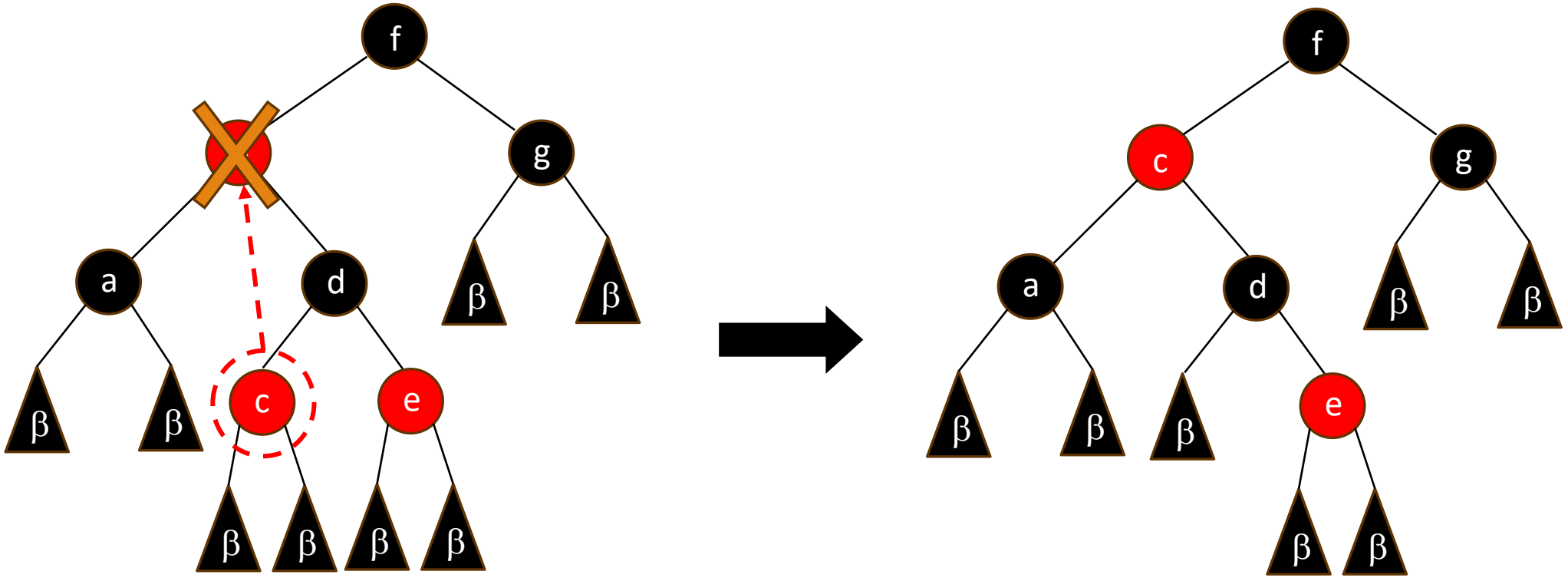
O problema da remoção efetiva é resolvido atribuindo negro a cor de ***c***.

Assim permanecerá igual a altura negra de todos os caminhos contendo ***c***, antes e depois da inserção.

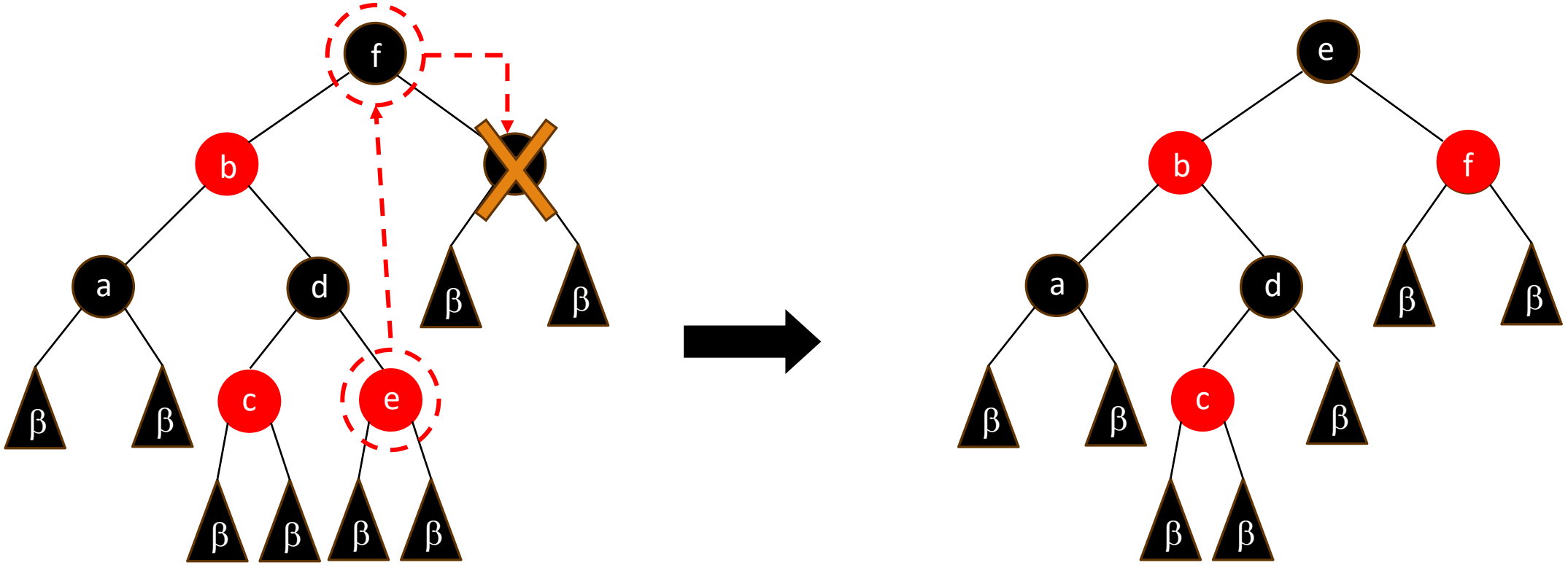
Remoção Efetiva



Remoção Efetiva



Remoção Efetiva



Comparações entre as Árvores AVL e Rubro-Negras

Árvore	AVL	Rubro Negra
Fator de balanceamento	Cada nó possui um campo ***bal***, que pode ser 0 (balanceada), 1 (desbalanceada a direita) e -1 (desbalanceada a esquerda).	Cada nó possui um campo ***cor*** que pode ser rubro ou negro.
Método de balanceamento	Se uma subárvore de um nó estiver 2 níveis maior que a outra subárvore (bal = 1 ou -1) ocorre uma rotação.	Caso haja dois nós rubros consecutivos ou a quantidade de nós negros até qualquer folha não sejam iguais ocorre uma rotação e, se preciso troca de cores.
Tolerância de desbalanceamento	Uma subárvore pode estar 1 nível maior que a outra subárvore de um nó	Uma subárvore não pode estar 2 vezes maior que a outra subárvore de um nó.
Crescimento	De cima pra baixo (raiz → folhas)	De cima pra baixo (raiz → folhas)

Simulador

Para visualizar mais exemplos, acesse:

https://www.inf.ufsc.br/~aldo.vw/estruturas/simulador/RB.html

Dúvidas?